REVON: LOG AND MERKLE DIFFERENCING FOR DURABLE STRUCTURED DATA VERSIONING

By Atharva Sheersh Pandey[1,a], Adhyan Jain[1,b], and Poornima Nedunchezhian[1,c]

1 School of Computer Science and Engineering, VIT Vellore

a atharva.sheersh2024@vitstudent.ac.in; b adhyan.jain2024@vitstudent.ac.in

c Corresponding author: Poornima Nedunchezhian (poornima.n@vit.ac.in)

Revon is a Python implementation of a key/value Merkle hash trie history, implemented on top of a SQLite database. We employ it for comparing log aggregation versus hash pruned diffing on one history, testing an operation-count based selection policy and measuring full versioning processes. For the Linux experiment, 516 runs come with an additional set of 72 runs geometry and telemetry information. In eight of nine timing comparison intervals for the hybrid/Merkle pairing, hybrid wins; the dense interval also includes parity. An additional threshold control shows a slower alternative: the policy selects Merkle if log is faster. The Windows experiment consists of 93 trials, each done in a new process, using disjoint sets of calibration and validation histories. It consists of controlled sensitivity analysis, Wine Quality and CO2 data sources, and repeated runs with 100,000 and 1,000,000 records. The winner of the calibration round uses log on every considered interval, so these results do not allow for concluding on the advantage of the switch. The CO2 histories include observed publication updates; wine corrections are artificial. Diff and historical state correctness are confirmed by semantic oracles; algorithmic, command line interface phase and full-process measurements are separated.

1. Introduction. An application that maintains dataset versions should be able to restore previous versions and determine the differences between them. Maintaining a full backup for each version partially solves this problem; however, the application also has to track and compare history at the record level. Git is known to use content-addressable objects for file history, where the content of objects uniquely identifies objects, and commits link parent objects [10]. These objects represent files and directories. Record history poses another issue. Keys could remain identical while updates are made more or less often.

We fix the history representation and compare two methods to traverse to its endpoint changes. The first method uses the operations recorded in the log, whereas the second compares Merkle hashes and skips identical subtrees. The two methods yield the same semantic difference starting from the same representation. To facilitate such comparison, we integrate existing techniques of path copying and content addressing with SQLite transaction model. Moreover, we estimate the effectiveness of a rule to choose which of the two paths to take. Timings include all versioning operations and data workloads considered in this work.

Other systems provide the context for such comparison. Dolt merges SQL database with commit graph and Prolly trees [13, 28]; ForkBase is an implementation of fork-aware database management system with POS-Tree and SIRI storage structure [32]; Noms

Keywords and phrases: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

describes history independent chunked collections [3]. Revon exposes key-value operations against a linear history to enable diff path comparison in a controlled manner. Such scope does not imply equivalence to feature sets of the aforementioned systems or introduction of new persistent indexes family. For cross-system measurements In our study, we employ Revon through its in-process API and Dolt through its CLI, separately recording the phases of the CLI for the clear boundaries of the comparison.

1.1. Research Questions. RQ1: What is the relationship between latency, persistent storage, and worker resources in versioning workflows, and how does the API/CLI distinction affect these measurements?

RQ2: How well does the selection of the paths by operation count succeed on generated histories not seen during calibration, compared to forced log and forced Merkle?

RQ3: Under experimental variations of tested histories, how do workload structure, payload size and fixed-trie geometry affect the cost of differencing?

1.2. Contributions. Our implementation uses atomic SQLite-based persistence, content addressing and path copying in the prototype consisting of two diff paths, both following the single contract for the encoded value. This integration and comparison of the paths under matched conditions represent the engineering contribution.

For the path selection we design a simple rule based on operation count and compare its choices with the forced execution of the paths. We complete the calibration process and register the selection threshold prior to testing on the held-out histories. Those cases when the selection takes more time than the paths execution are part of the analysis, as well as the question about using the log for all of the histories.

The verification evidence is based on the comparison of the observed diffs and historical states to the oracles constructed independently from the storage implementation. The executed sources, the raw measurements, the environment configuration and data preparation procedures are attached with the paper. This also includes the description of the correctness checks performed and what is measurable by the telemetry.

We provide the results of the Linux workflow experiments along with 93 case Windows evaluation campaign. In our Windows setup, validation seeds are distinct from calibration seeds; matched sensitivity control, Wine Quality and CO2 sources, multiple scales runs extend the comparison. Since each experiment employs a different experimental unit, each campaign is analyzed separately, and their timing rows are separated.

The evidence scope is limited by the single writer store, linear histories and the tested workloads. It can provide the trade-offs in these conditions. It cannot determine the universal best selector and geometry, prove query engine superiority or reproduce with another team independently.

2. Related Work.

2.1. Content Addressing and Persistent Structures. A collection authenticated using its Merkle root can be authenticated using that root [22]. Content-addressing allows software history in Git and IPFS in the case of networked content [10, 5]. Persistent data structures use path-copying to keep old versions and share the unchanged parts of the data structure [14]. Ordered stratified B-trees [31] and fully persistent B-trees [8] solve disk-based dictionary problems. Revon employs an alternative routing technique, where hashes are passed to a fixed-depth trie. Matching hashes can stop a comparison but cannot be used for an ordered range search. SciDB and TimeArr are designed for versioned arrays having layouts and deltas designed specifically for arrays [27, 30]. TardisDB provides versioned

accesses in main memory, and Saquib et al. study replicated versioned data structures in wide area networks [25, 26]. These designs may give us some clues about the design decisions made in Revon, but they do not cover the comparison between the log and Merkle paths of Revon in the same history.

Merkle Search Trees resolve states by merging search tree ordering and Merkle authentication [4]. In Revon, search is pruned by hash, but the routing strategy is fixed-depth, and history is linear and central as opposed to replicated CRDT. LBFS detects redundant content based on content-defined chunking [23]; there are other chunk boundary techniques in Dolt and Noms. Chunk boundaries allow for deduplication during movement of data and ordered indexes for range searches. Neither of them is covered in Revon benchmark.

2.2. Structured Dataset Versioning. A system for dataset-versioning needs to enable collaboration in addition to state persistence and reconstruction. DataHub explores collaborative histories at scale [6]. Decibel facilitates relational branching using version-first, tuple-first and hybrid representations [20]. OrpheusDB relies on partitioning to enhance version lookup if an already extant DBMS becomes capable of versioning [19]. The storage-reconstruction dilemma is addressed by Bhattacherjee et al. [7]. Delta Lake creates ACID-compliant tables via transactions logs and checkpoints [2] while the version graph approach picks materialized versions and deltas under storage and retrieval constraints [18]. FAIR-compliance related problems of release naming and description are the topic of the FAIR-compliant proposal [16], which adds to the current storage research without providing any diff algorithm. Revon solves the limited version-graph part of this more general problem with a key/value interface.

2.3. Design Gap Between Systems. Noms is a database storing Merkle DAG of structured values along with diff and sync capabilities [3]. ForkBase is a combination of fork-and-dedup semantics and content-addressing [32]. In Dolt, content-addressed Prolly Trees serve as the basis of structural sharing, native diff, and ordering indexes [13, 28]. Git4Data provides snapshot, branching, diff and merge capabilities in SQL over a relational engine [17]. This work is mentioned as the arXiv preprint, not peer-reviewed results. Revon is a simpler system with a focus on linear history and fixed depth of hash routing in a single-writer key/value database without SQL or branching.The systems and studies most relevant to these implementation decisions are collected in Table 1.

Table 1
Related systems and studies

System or study	Representation or focus	Relevance to Revon
Persistent DS [14]	Path copying	Formal structural-sharing basis
Git [10]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [5]	Merkle DAG	General content-addressed versioning
MST [4]	Merkle search tree	Authenticated ordered reconciliation
DataHub [6]	Version graph	Collaborative dataset motivation
Decibel [20]	Relational layouts	Native database branching
OrpheusDB [19]	Partitioned versions	Storage/retrieval optimization
Delta Lake [2]	Log/checkpoints	ACID time travel at data-lake scale
Noms [3]	Merkle DAG	Structured content addressing
ForkBase [32]	Forkable CAS	Fork and deduplication semantics
DoltHub [13, 28]	Prolly Trees	Mature production comparator

2.4. Design Implications and Research Positioning. Representation choices move work between version creation, retention, reconstruction and comparison. A complete snapshot is easy to retrieve but copies unchanged records. A log leaves work for replay or checkpointing. With path copying, the unchanged structure is shared and only update-affected paths are rebuilt [14, 7]. Decibel and OrpheusDB show how layout affects version creation and retrieval [20, 19], and version-graph analyses make the storage/retrieval trade-off explicit [18]. Reproducibility research considers how builds and artifacts can be kept available for inspection [1]. We therefore report import, commit, checkout, diff and storage separately. Retaining an artifact enables inspection; it is not evidence that the implementation runs faster.

Different studies also mean different things by a change. Differential dataflow updates query results when timestamped inputs change [21]. Provenance describes the origins of output facts and why those facts occur [9]. DSLog captures fine-grained array lineage and supports queries over it [33]. Explain-Da-V relates semantic dataset changes to transformations [29], while Baseline uses operation-based differencing for refactorings and schema changes [15]. Revon restricts its output to keys added, removed or modified between two states. Causes, lineage, effects on queries and schema intent are not inferred. The related studies provide context, but their workloads and outputs are too different to support direct timing comparisons with these key/value diffs.

Each Revon-H commit retains both a content-addressed changeset and a Merkle root, allowing either path to recover the same endpoint changes. The selection rule changes how the diff is computed. We expect log aggregation to suit short, sparse intervals and hypothesize that hash pruning may help after longer sequences of operations; whether their costs cross has to be measured. Git4Data shows the wider interest in database-native version control [17]. For our fixed-depth-trie comparison, Snapshot and Log-only serve as representation baselines. The public workloads consist of generated TLC and wine corrections and observed changes in a CO2 publication sequence. Their results do not

establish performance across independent hardware, concurrent workloads or production systems.

2.5. Capabilities and comparison boundaries. Table 2 distinguishes what the systems document from the evidence used in this paper. Feature descriptions are not benchmark results. ForkBase and Noms describe typed-collection APIs, but we evaluate no relational layer over either. The Git4Data preprint describes a different abstraction level [17]. A capability left unevaluated here should not be interpreted as absent from the system.

Table 2
System capabilities and evidence scope; documented features are not performance results

System	Representation / history	Diff / persistence	Branches / queries	Reproducibility and use here
Revon	Fixed-depth hash trie; linear history	Log or Merkle; SQLite objects	No named branches/merge; key/value API	Executed snapshots, semantic proofs and raw trials; measured here
Dolt [13, 28]	Prolly Trees; commit graph	Native version diff; persistent block store	Branches/merge; SQL	Official docs/source; CLI workflow measured, engine not isolated
ForkBase [32]	POS-Tree/SIRI; forkable versions	Hash-pruned diff/merge; addressed chunks	Fork on demand/conflict; typed KV APIs	Primary paper and APIs; no experiment run here
Noms [3]	History-independent chunks; Prolly collections	Hash-based diff/sync/merge; chunk store	Typed collection API; not benchmarked here	Technical overview/source; conceptual comparison
Git4Data [17]	MatrixOne immutable objects/MVCC; table versions	Database-native diff/merge	Snapshot/tag/branch ; SQL extensions	Preprint/BranchBench claims belong to its authors; not run here

2.6. Related foundations and scope boundaries.

Research-data versioning guidance includes version identity and descriptions of change as requirements for reliable citation and reuse [34]. The IPFS evaluation and FastVer examine content-addressed persistence and integrity beyond the setting of our local prototype [35, 36]. Prior states can also be retained and verified with persistent tries, B+-trees, external-memory search trees, snapshot stores and authenticated data structures [37-41]. Repository views, collaborative analytics and DEX illustrate how history systems support access to repositories and collaborative work [42-44].

Schema-evolution and temporal-database research addresses changes in the structure and meaning of data over time [45-47]. Work on provenance traces where results come from and how they are derived, using semirings and accounts of scientific workflows as well as tutorials and broader syntheses [48-51]. Further studies cover aggregate-query provenance, authenticated or secure provenance, FAIR stewardship, privacy and the PROV model [52-56]. In SEEK, provenance and data management are brought together within a scientific repository [57].

Several storage designs offer context for our tree and log mechanisms. Foundational B-tree work and cache-oblivious indexing explore different search and update costs [58-61]. LSM trees, Bigtable, Dynamo and log-structured file systems are examples of log- or append-oriented storage [62-65]. WiscKey and LogBase study write-efficient key-value and log-

based database storage, and multi-version partitioned B-trees handle version-aware indexing [66-68]. They motivate comparisons of design choices, but the workloads and measurements in those studies cannot replace the ones evaluated here.

3. Revon Design. Figure 1 shows the implemented architecture and places the evaluated operations within it.

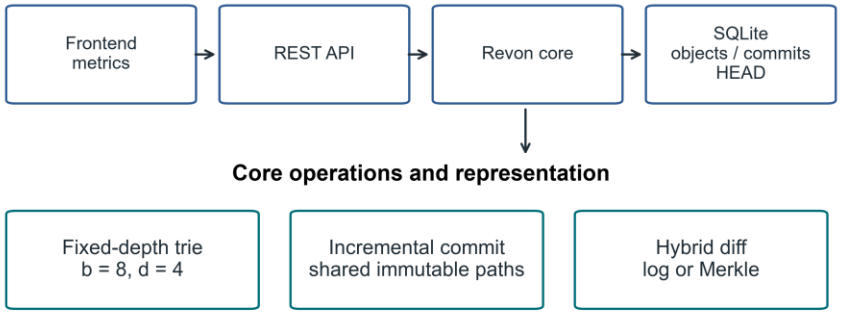


Fig 1. Revon architecture showing the implemented scope of the research prototype

3.1. Addressed Object Model. Revon names a trie node or changeset by the SHA-256 digest of its canonical JSON serialization. The digest for a commit includes the root, parent, changeset, message and timestamp. Human-readable names such as v1 and v2 are aliases and do not affect object identity. The current commit is identified by a durable HEAD reference.

3.2. Fixed-Depth Merkle Hash Trie. For the main configuration, b=8 and d=4. Each routing level consumes three bits of SHA-256(key), giving twelve bits that select one of 4,096 possible leaf buckets. Leaves retain complete keys, so keys sharing a routing prefix remain distinguishable. Expected occupancy under uniform hashing is N/b^d. Other b,d combinations are evaluated separately; we do not assume the default minimizes cost.

3.3. Incremental Hashing and Commit. A batch is processed by canonicalizing its values and grouping keys with the same routing prefix. Revon rewrites each touched leaf once, then rebuilds the ancestors on the affected paths. Unchanged child hashes remain in internal nodes, sharing the structure beneath them. The objects, changeset, commit metadata, version alias and measurements are written and HEAD is updated in a single SQLite transaction. Reopening the repository checks stored payload hashes and references.

3.4. Hybrid Diff. Log differencing composes operations by key while traversing the requested ancestor interval. Merkle differencing compares roots recursively, stops when hashes match and examines entries beneath unequal paths. Revon-H validates the version identifiers, checks ancestry in both directions and counts changeset operations before selecting a path. Log is used for an eligible ancestor interval whose count is no greater than T; otherwise selection goes to Merkle. A reverse log request exchanges old and new values, while a request comparing a version with itself returns an empty diff. Forced log rejects unrelated versions. Unknown identifiers raise an error. Revon-M always requests Merkle, with ancestry still checked by its public version-pair API. Both paths produce the same sorted semantic diff, and commit and checkout are timed independently.

The Linux workflow protocol sets T=4,096. For a separate Linux control, an auxiliary calibration experiment selected T=16,384. T measures operation count, whereas b^d determines bucket capacity. Neither parameter was fitted to the Linux results.

3.5. JSON value ownership and identity. Revon accepts JSON nulls, booleans, finite numbers, strings, lists and objects with string keys. Input containers are validated and detached before storage, and containers returned through reads or diagnostic views are detached too. Immutable scalars may be shared. Before writing, Revon rejects unsupported Python values, nested non-string keys, non-finite numbers and cyclic containers. Its v1 JSON encoding defines equality through sorted object keys, compact separators, UTF-8 and the exclusion of NaN. The encodings of true, 1, 1.0 and signed floating zero remain distinct. These requirements describe the encoding contract of the Python prototype, without asserting compliance to the canonical-JSON encoding of cross-language JSON.

Algorithm 1 (batch update). Start by verifying the batch and claiming the values. Prior to interning an object, reject keys appearing in both puts and deletes, and requests to delete non-existent keys. Read the old values and keep the mutations if the encoded identity has changed. Group the mutations by routing prefix, reconstruct each affected leaf and sort its entries and hash the entire payload. Recreate the affected ancestors with the unmodified child hashes wherever possible. Log the content-addressed changeset and commit. Detaching values stops users from changing history through their input containers and read operations.

Algorithm 2 (log composition). Scan the ancestor interval chronologically. For each key, keep its existence and value at the earliest old and latest new endpoints. Remove it from the results if the two endpoints have equal existence and value. The insertion followed by deletion cancels out, as does the update followed by restoration. Null is different from non-existence. Sort the keys left in the results; reverse requests switch endpoints and the added/deleted labels.

Algorithm 3 (selection). Verify the version pair and check whether one is an ancestor of the other. Sum the operation counts of changesets in the interval. Pick the log if the interval is valid and the sum is less than or equal to T; otherwise pick the Merkle tree. Merkle traversal skips the trees with equal hashes and compares values in the unequal leaves. Apply the encoded identity test to both, detach values returned to the caller.

Cost model. Let N be the record count, d the depth and b the branching factor. Write m for the keys in a batch, h for interval length, p for accumulated operations, u for distinct keys in the interval and s for serialized output bytes. Expected leaf occupancy is N/(b^d) with uniform routing, but can reach N in the worst case. Besides validating and copying inputs, a batch performs m routed old-value lookups and scans occupied leaves. It sorts and serializes touched leaves and copies affected internal child maps. This work prevents a dataset-independent O(d) bound for batch processing.

Log composition traverses ancestry with O(p + u log u) work, plus the costs of encoded-value comparison and output copying/serialization. Merkle cost depends on the visited nodes pair, child-slot sorting, and leaves examined. Value comparison and output sorting also contribute to Merkle cost. In both public version-pair modes, Merkle tree ancestry traversal takes place. If the request is reversed, the unsuccesssful direction will revert to genesis first. Construction of the output will take at least linear time of s. Comparison and copying of large JSON values require additional cost than the benchmark of scalar value can provide.

Retained nodes, changeset, and commit are read eagerly from the SQLite database. Even if they are durable on the disk, they are kept in memory as long as Revon runs; thus, retained history needs to be able to fit in memory. Retained payload will affect the performance of repository open and validation process. No disk page caching or eviction strategy is included in this prototype.

Figure 2 traces the selection process for a version pair, including eligibility checks and reverse requests.

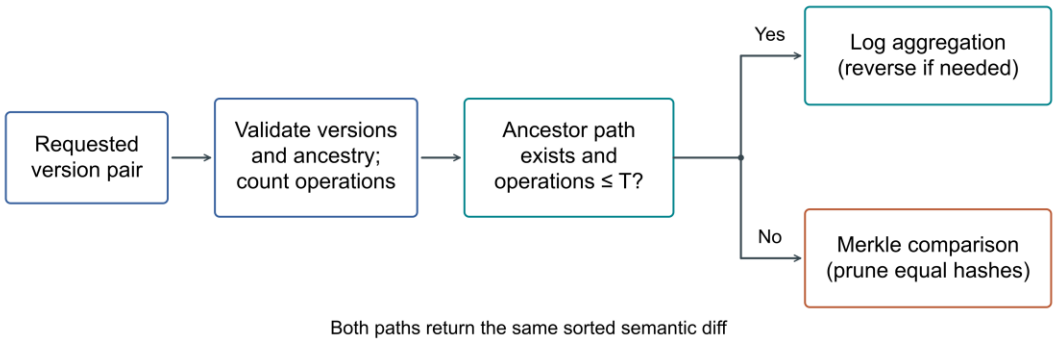


Fig 2. Diff-path selection for a version pair. T gives the operation threshold. Invalid versions raise an error before a path is chosen, and both paths use the same persistent trie

4. Experimental Methodology.

4.1. Environment and evidence boundaries. The Linux experiments ran on an Ubuntu n2-standard-4 cloud VM, whose environment and executed sources appear in Table 3. Windows experiments ran on a separate computer operated by the authors. The authors conducted both campaigns. Since hardware, OS, runtime and storage change together between them, differences cannot be attributed to any one factor.

Table 3
Linux execution environment and the recorded sources

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: executed source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

The campaign records include executed-source hashes, a fixed protocol, environment information, correctness checks and raw observations. Linux preflight suites passed 67 and 69 tests; the Windows suite passed 77. Setup failures and the public-data exclusion applied before timing remain documented. Appendices A and B give access information and detailed procedures.

4.2. Variants and workloads. We measure six workflows across five systems. Snapshot stores a complete state at each version, whereas Log-only stores an initial state followed by operations. Revon-M requests Merkle diff; Revon-H chooses the path. Dolt stores records in a primary-key SQL table and produces its native JSON diff. SQL INSERT and CSV table import are timed as separate Dolt workflows. Subsequent operations conform to a shared semantic-output contract despite differences in interfaces and storage features. The figure workload IDs are defined in Table 4.

Table 4
Linux workloads, each primary history having ten generated commits

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

The eight synthetic scenarios account for 432 executions: each has six variants with two warm-ups and seven measured trials. Public-record workloads contribute 27 executions for Revon-M, Revon-H and Dolt SQL. Threshold controls contribute 54, and the three million-row diagnostics bring the total to 516. This comprises 114 warm-ups, 399 measured trials and three diagnostics. All executions finished with verified output. No failed timing execution is silently replaced or excluded from the accounting.

4.3. Public dataset and generated histories. TLC input comes from the January 2024 yellow-taxi release [24]. A checksum adjusts the Parquet input to 2,964,624 rows. Using seed 20261001, a dispersed affine permutation produces nested subsets containing 10,000; 100,000; and 1,000,000 rows. Eleven fields are kept: vendor, times of pickup and dropoff, number of passengers, distance, locations of pickup and dropoff, method of payment, fare, tip, and total. Key fields are release number, file-checksum prefix, and physical row ordinal. Keys refer to rows in the particular file, and not trips across releases. Values, including NULLs, are serialized into compact UTF-8 JSON with sorted keys. All rows serialize correctly, including 47,277 missing passenger counts in the 1,000,000 row subset. Identical values do not make rows merge. The affine process does not uniformly randomly sample.

Ten commits are produced for each TLC history. In the Linux 100,000-row workload, each commit changes 100 records; in the 1,000,000 row diagnostic, it changes 1,000 records. The correction increments the fare and total fields by 0.01; all other kept fields are unchanged. Inputs and mutations are identical in all three systems. These are generated corrections and not trip modifications. Public data builder ignores the inherited payload_bytes = 8 configuration field, keeps all eleven source fields, and uses measured size of the serialized value. Canonical records are encoded as JSON strings in the common value field. Therefore, measurements apply neither to native nested-object modification nor separate field queries.

4.4. Timing, correctness and uncertainty. Preparation and repository construction precede the timed operations, followed by correctness verification and analysis. We give import, commit, diff and checkout separate timing boundaries. Under the common output contract, diff returns sorted keys and hashes of their old/new values. Table 5 sets out which measurements cover algorithm work, interface costs or the whole worker. Correctness checks and decision probes are outside the operation timers.

Table 5
Timing boundaries and interpretation of the measurements

Quantity	Revon / same-representation control	Dolt CLI workflow	Interpretation
Diff latency	Version API, selected path and canonical output	Native JSON-diff command, parsing and canonical output	Complete interface contract; no engine-only ranking
Phase probe	Separate ancestry/count probe; includes detached-view copying	Command, JSON parse and output-construction timers	Neither probe nor command isolates pure tree-engine work
Correctness	All states, reverse/identity, integrity and reopen outside timers	Observed native rows and historical checkout outside timers	Observed values compared with an independently constructed oracle
Resources	Worker plus sampled children, including oracle/setup	Worker plus sampled Dolt processes	10 ms samples; short processes/peaks may be missed
Persistent bytes	Windows: closed files; Linux: before close/compaction	Repository directory files	Includes metadata/indexes; not isolated algorithm overhead

We check the observed endpoint diff and every historical state against workload oracles constructed separately from storage code. Revon also checks reverse and identity requests, object integrity and repository reopening. Reconstructing the plan and semantic hashes lets the audits establish internal consistency. It does not repeat the timing experiment.

For each fixed Linux history, seven trials create fresh repositories, with paired percentile-bootstrap intervals used for timing estimates. A Windows case instead makes seven cached queries within one fresh repository. Its medians across generated histories are reported separately from those across repository repetitions. Those are different experimental units, and neither campaign samples a population of machines.

Sampling covers setup, verification and storage operations for the worker and its observed descendants. CPU and memory measurements thus include work outside the storage implementation. Primary Linux read-counter records do not distinguish supported zero values from missing counters. The Linux supplement and Windows sampler do distinguish them. Windows process I/O bytes nevertheless cannot measure direct physical-disk traffic.

Repository footprint includes persistent metadata and representation costs. Linux directory size is recorded before closing or compacting. Windows size is recorded after closing and the specified verification steps. These conditions differ, so we report the measurements separately; neither isolates the tree's overhead.

4.5. Frozen threshold and geometry controls. The auxiliary Windows calibration selected its largest candidate, T=16,384, without establishing a crossover. The Linux controls use that threshold unchanged. A separate Windows selector experiment, described below, has its own fixed design, executed source and disjoint calibration/validation seeds. Its threshold results are analysed separately.

At 10,000 rows, the Linux control compares forced log, forced Merkle and hybrid with T=16,384 over intervals of 8,192 and 32,768 accumulated operations. This control is not chosen or retuned from Windows outcomes.

The Linux geometry comparison uses one fixed spread workload and five configurations. The Windows sensitivity comparison uses three generated histories and matched equal-capacity geometries. Bucket capacity and operation threshold are distinct parameters. Results describe the settings tested without establishing a geometry that is generally optimal.

4.6. Windows calibration and validation. In the Windows selector comparison, N=12,000, H=8, payload=32 bytes and b=8,d=4 are fixed. We use two calibration seeds and three different validation seeds, crossing spread/repeated-key histories with 64, 512 and 4,096 changes per commit. Each case shares a durable representation across forced log, forced Merkle and hybrid thresholds T=512/4,096/16,384/65,536. Deterministically ordered configuration. Two warm-up runs followed by seven runs for evaluation of the query blocks are done. Every scenario is started from a new worker, but its queries act on common resident objects and use the file system cache. These results cannot be considered for cold-cache situation.

Calibration is based on calibration of the median of every hybrid scenario by median of the corresponding lower forced-path scenario. Then equally weighted average is minimized with smaller T value in the case of ties. The threshold and UTC selection time for it is measured before start of the first validation worker because validation cannot choose another winner. In the sensitivity analysis the number of rows, history interval, payload, geometry, and routing are varied step by step according to the predefined scenarios. Since the number of accumulated operations is fixed, the history interval is related to changes per commit and, hence, full-factorial design cannot be used here.

4.7. Public histories and repeated scale experiments. The Wine Quality dataset is composed of 6,497 records about red and white wines; keys are created based on the color of the wine as well as its ordinal position within the source record [11]. For every three histories produced, there are eight runs of 64 seeded corrections for the quality attribute. These corrections do not capture any change made to the actual specimen. The CO2 package supplies updated CSV files that are made available through sources derived from NOAA [12]. Before freezing the experiment, there are eleven snapshots captured in the last that made changes to the file; one snapshot of failed publication containing only the header is not included. There are ten snapshots left, resulting in nine transitions out of 812 records. Unnamed trailing columns are maintained to be another field, and dates are used to distinguish records. The order and format of CSV files do not affect state comparison.

The three CO2 repositories repeat the same observed sequence. Wine and scale repositories use distinct generated seeds instead. We run each public source through Revon-M, T=4,096 Revon-H and native Dolt JSON diff, rotating model order. For scale, H=4, C=64 and payload=32 are fixed at 100,000 and 1,000,000 records, with three seeds per size/system. We report descriptive medians and full ranges across repository medians. The metadata retains source URLs, licences, acquisition versions, checksums and

transformations. Restricting acquisition to a bounded publication history made it feasible on the available host without a bulk Software Heritage collection.

The execution-count matrix is in Appendix B, Table 17; the TLC checksums and Linux public-data protocol are in Table 16. The main text supplies timing boundaries and experimental units so that interpreting results does not require reconstructing the entire execution manifest.

5. Linux Results.

5.1. Hybrid versus forced Merkle. Revon-M and Revon-H have equal persistent data structure and change-set representations. In the current experiment, the difference strategy being used is the only varying parameter in the ablation. All nine cases of repeated trials are shown in Figure 3 and Table 6. The hybrid interval does not use parity in eight cases; for dense S3 it uses parity in one case. Log access helps the sparse and repeat key history being analyzed; but this is relevant to the performance of diff process and not decrease in data storage.

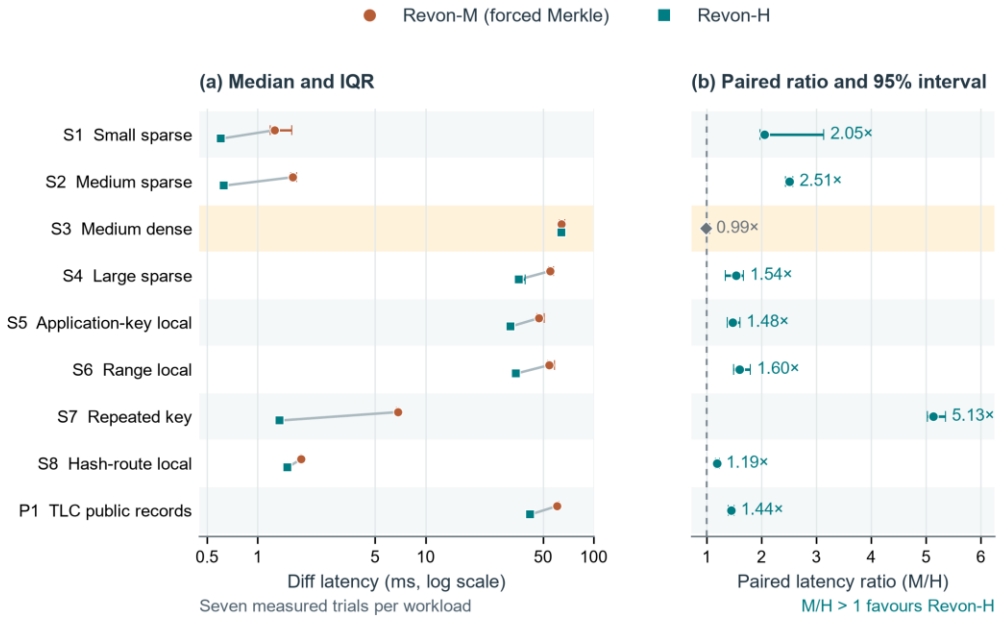


Fig 3. Linux comparison of Revon-M and Revon-H diffs. In (a), connected workload pairs show median latency and interquartile ranges on a logarithmic axis. In (b), median matched-trial M/H ratios have paired percentile-bootstrap 95% intervals from 20,000 resamples. The dashed line is parity. Seven measured trials contribute to each workload. S3 is highlighted because its interval contains parity; the remaining eight favour H. The median of trial ratios need not match the ratio of median latencies. Table 4 gives workload definitions. The intervals describe repeated timings for fixed histories on one VM, not variability between independent datasets or machines.

Table 6
Paired M/H diff ablation, with latency in milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

P1 has median diff latency of 41.964 ms under Revon-H and 60.597 ms under forced Merkle, using identical eleven-field public records and generated edits. On S3, H chooses Merkle for the interval's 10,000 accumulated operations. Its near-parity timing is consistent with a shared diff path and the extra selection work in H.

5.2. Threshold control and adverse result. Figure 4 and Table 7 retain both controls, including the slower selection. Hybrid uses log for 8,192 operations and switches to Merkle for 32,768. Forced log is faster in all seven matched measured trials at the larger count. For these cases, operation count alone therefore fails to choose the fastest path. This Linux test of the fixed threshold leaves the best threshold undecided.

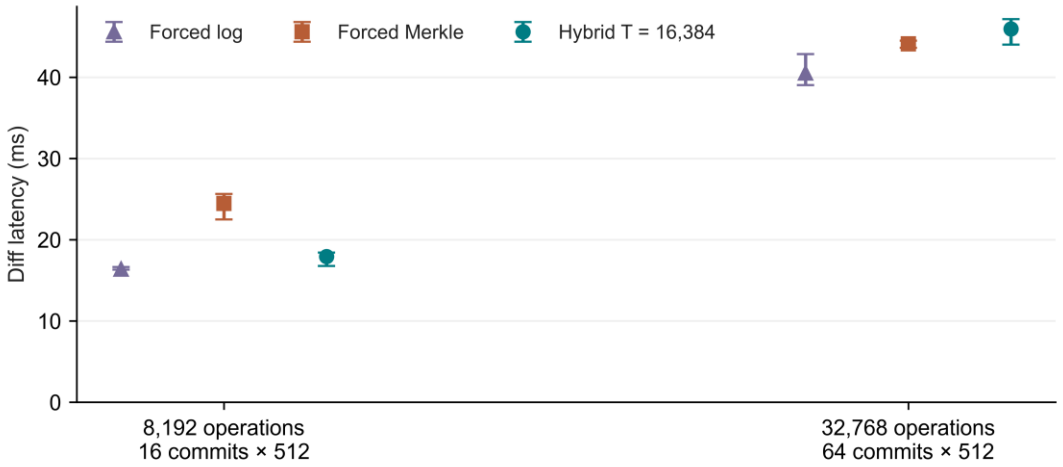


Fig 4. Fixed T=16,384 policy in the Linux controls. Points are medians and bars are IQRs from seven measured trials per path; the labels identify history lengths

Table 7
Diff medians in milliseconds for the fixed-threshold controls

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

5.3. Complete workflow comparison. Canonical-diff procedures have been compared in Table 8 through the use of the API provided by Revon-H and the SQL/CLI interface of Dolt. Every single one of the nine pairs of intervals of Dolt and H is greater than one. The times include process startup time, data structures, programming language issues, serialization, querying, and in-memory state, and therefore, the efficiency of the storage engine cannot be separated from the ratios given.

Table 8
Dolt SQL/CLI and Revon-H diff latency in milliseconds, with paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

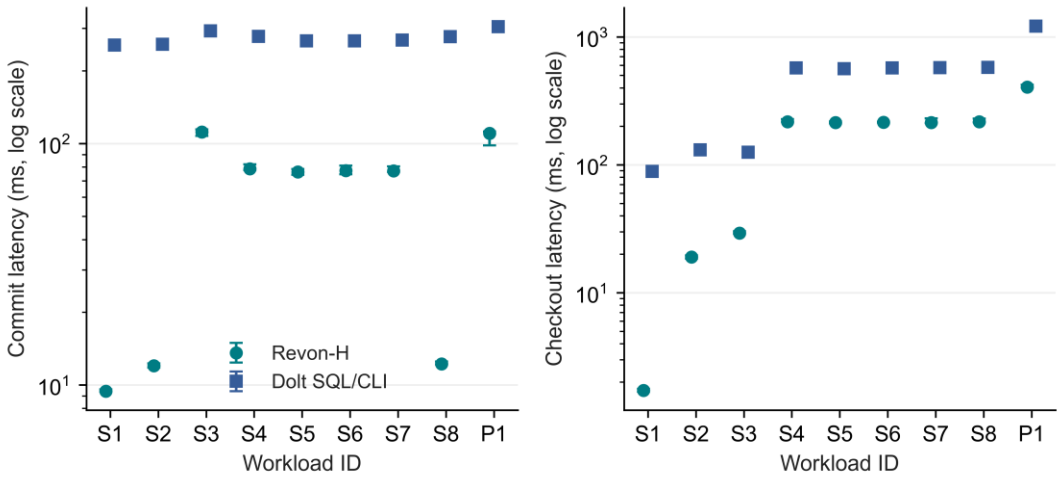


Fig 5. Linux incremental-commit and full-checkout timings for Revon-H and Dolt SQL/CLI. Points represent medians and bars represent IQRs over seven trials, with logarithmic latency axes

5.4. Storage and memory costs. Figure 6 and Table 9 report persistent directory size and whole-worker peak RSS alongside latency. M and H both retain content-addressed changesets and trie data. Selecting log removes no data from that representation; the small directory-size difference comes from the measured workflows. Dolt also uses different serialization, compression and indexing. Lower diff latency can thus come with a substantially larger repository. An efficiency assessment has to include storage and worker resources as well as latency.

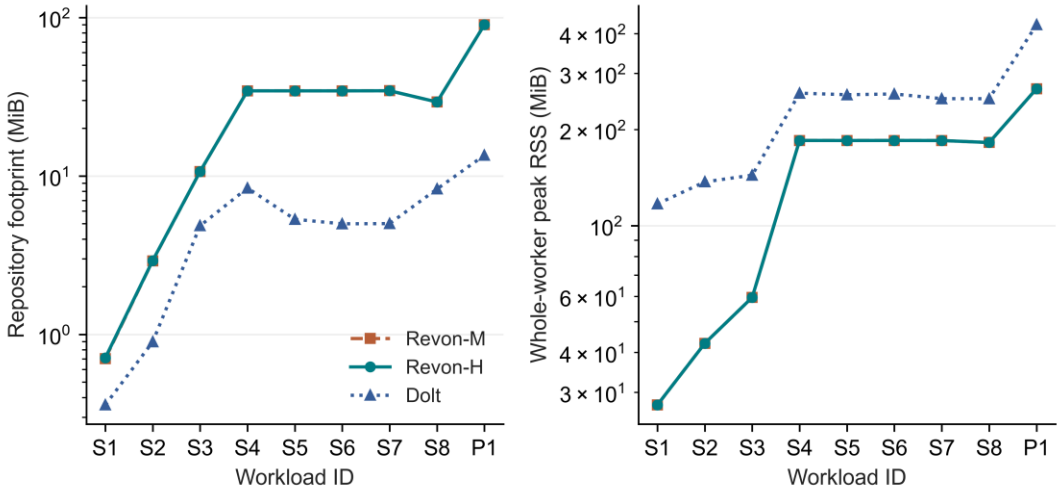


Fig 6. Median pre-compaction repository size and median peak RSS for the whole worker over seven Linux trials, plotted on logarithmic axes. RSS includes shared workload and verification memory

Table 9
Median repository footprint and worker RSS in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

5.5. Million-record feasibility. The Linux million-row diagnostic creates and verifies one fresh repository for each of M, H and Dolt. Verification covers all eleven historical states and the shared diff oracle. The individual observations are listed in Table 10 without IQRs, confidence intervals or rankings. Hybrid selects Merkle because 10,000 accumulated operations exceed T=4,096. These runs demonstrate completion of the bounded workload on the VM, but one observation per system cannot determine relative scaling or an optimal configuration. Section 6.4 presents the separate repeated Windows scale experiment.

Table 10
Single million-record observations, n=1 per system, without comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

5.6. Fixed-trie geometry and telemetry. The Comparison of Linux Geometries is shown in Figure 7 and Table 11. In each case where b4-d6, b8-d4, and b16-d3 are used, there are 4,096 leaf buckets. In this case, the same expected occupancy can be used to evaluate the routing shape at such capacity. Keeping the value of b constant at 8, different depths result in a different capacity and balance between traversals and leaf bucket scans. Since one workload is used, a general optimum geometry cannot be determined.

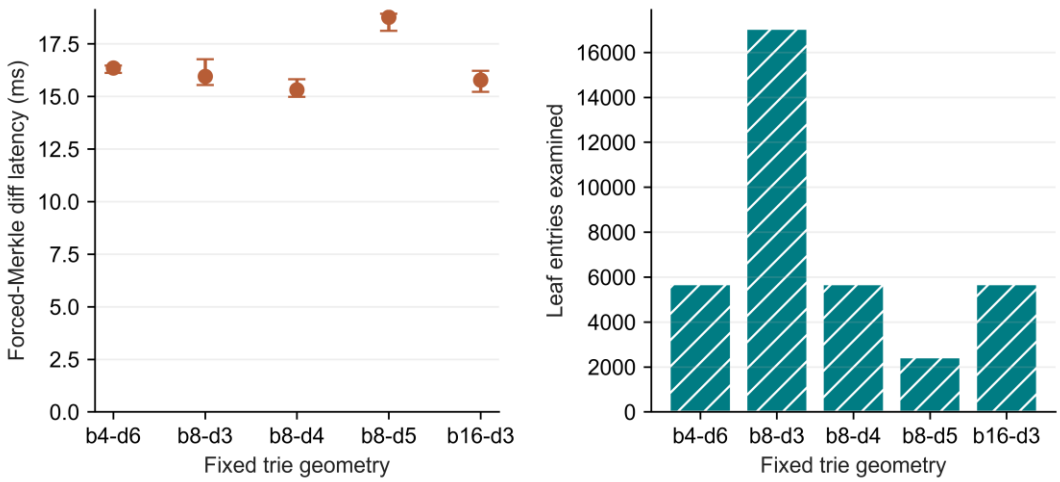


Fig 7. Fixed-trie geometry measured on Linux. Left: forced-Merkle diff medians and IQRs over seven trials. Right: median examined leaf-entry count

Table 11
Work and footprint across geometries for one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

Both isolated worker telemetry test cases work based on the distinction between all zeros that are true input/output results and all zeros indicating unavailability of the counter. The Linux telemetry extension is performed using 27 repetitions for 10,000 rows with M, H, and Dolt having 2 warm-ups and 7 test executions. Measured values of zero read bytes stay recorded as such, without empty entries. This check ensures correct counter processing, but no cold storage is observed and there is no proof that no measurements were lost.

5.7. Representation baselines. Table 12 uses representative spread workload S4 to give descriptive medians for all six workflows. The GitHub reproducibility archive described in Appendix A provides the full eight-scenario matrix, dispersion between trials, normalized storage and separate SQL/CSV import workflows. Snapshot and Log-only are representation controls in the common harness, not equivalents for all production features. Their full-state storage and operation-replay costs put Revon's measurements in context.

Table 12
Representative S4 workflow medians; the GitHub reproducibility archive supplies the complete eight-scenario matrix

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6. Windows Evaluation Results.

6.1. Held-out selector behaviour. Calibration selects the largest candidate, T=65,536, at a normalized score of 1.014. Across 18 held-out histories its mean normalized latency is 1.012, compared with 1.101 for T=4,096. The respective agreements with the lower forced-path median are 18/18 and 12/18. These counts refer to noisy measured medians, not classification against a known optimum. Table 13 and Figure 8 retain every profile group.

Primary Windows intervals contain no more than 32,768 operations. As a result, T=65,536 uses log everywhere in this matrix. Its calibration win does not demonstrate useful switching or establish a global crossover. Timing noise can also produce ratios below one when hybrid and forced execution choose equivalent paths. This result, together with the adverse Linux control, bounds the conclusions about the selector. The threshold selected for the experiment leaves the implementation's default unchanged.

Table 13
Held-out median of three history medians at fixed N/H/payload; no population interval

Locality	Operations	Forced log ms	Forced Merkle ms	H4096 ms	Calibrated ms
spread	512	1.913	4.622	1.907	1.837
spread	4,096	25.714	38.878	26.550	27.312
spread	32,768	92.792	104.71	106.56	91.775
repeated-key	512	0.564	1.661	0.549	0.573
repeated-key	4,096	5.608	14.342	6.311	4.664
repeated-key	32,768	62.290	74.366	77.092	67.721

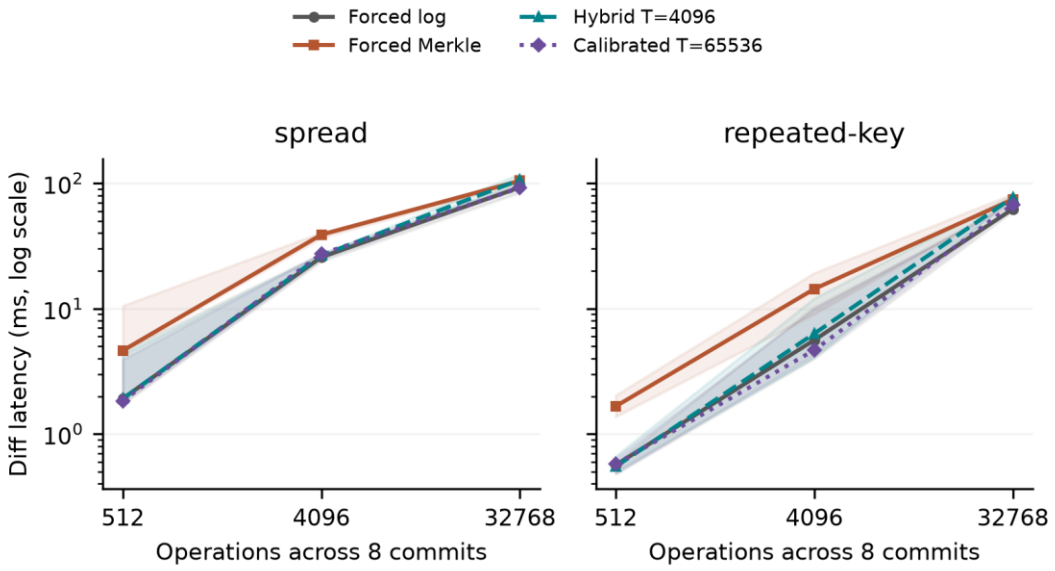


Fig 8. Selector results on held-out histories at N=12,000 and H=8. Lines link the medians of three generated-history medians; shading extends from the minimum to the maximum. Each history provides seven measured cached queries. The shaded range is descriptive, not a 95% confidence interval. The calibrated policy selects log at all plotted operation counts. All forced paths share the durable representation.

6.2. Sensitivity and resource interpretation. This decision probe runs outside the differential timing window and uses an independent public changeset view. Such copying overhead makes the probe incapable of measuring the selector overhead independently. Another diagnostic technique is used for obtaining the production-function prefix for validation, ancestry evaluation, and internal operations count until dispatching ends. From 18 held-out histories and using two thresholds, 252,000 warm calls lead to a median of the medians per case and threshold of 2.674 microseconds with a range of 1.611 to 3.678. The decisions are consistent with production dispatching. The diagnostic purposefully avoids differential traversal and copying overhead, and neither measures cold-cache performance nor a correction that can be subtracted from the workflow timings.

Across three seeds, the 21 sensitivity cases vary a prespecified factor or a matched control. They include payload=256 bytes, N=24,000, fixed-operation H=16, equal-capacity b4-d6/b16-d3 geometries and a C=8 hash-route/spread pair. The geometry cases change routing geometry while reproducing the base history's state hashes. With operations held fixed, history length trades off against changes per commit, preventing estimation of their independent effects in that comparison. The results describe these profiles and do not choose a generally best geometry.

Median forced-Merkle latency is 47.857 ms for more rows, 40.122 ms for longer histories and 39.950 ms for larger payloads. The b4-d6 and b16-d3 geometries give 43.004 ms and 38.500 ms, respectively. Hash-route locality gives 0.174 ms compared with 0.903 ms for the matched spread control. Each value summarizes three history medians by their median. Per-history timing, selected paths and resource scopes are retained in the summaries. We treat these comparisons as exploratory and make no multiple-testing significance claim.

For T=4,096, median-latency ratios in the three sampler off/on pairs range from 0.950 to 1.206. Sampling-off runs always come before sampling-on runs, with cache state and background activity uncontrolled. This ordering prevents a causal estimate of the sampling penalty, and we subtract no correction from other timings. Resource counters measure

entire workers. Although tests preserve observed zero I/O as a value, Windows counters cannot identify database logical reads or direct physical-disk traffic.

6.3. Public data and interface phases.

Table 14
Wine and CO2 workflows: medians and full ranges of three repository medians; one observed CO2 history is repeated

Source / system	Runs	Diff ms	Range ms	Import s	Persistent MiB
co2/revon-m	3	0.612	0.562-0.661	0.033	1.191
co2/revon-h	3	0.309	0.302-0.318	0.035	1.184
co2/dolt	3	256.98	251.87-262.89	0.634	0.178
wine/revon-m	3	4.281	4.260-4.964	0.189	10.832
wine/revon-h	3	2.247	1.956-2.366	0.191	10.812
wine/dolt	3	296.76	246.75-302.81	0.819	1.360

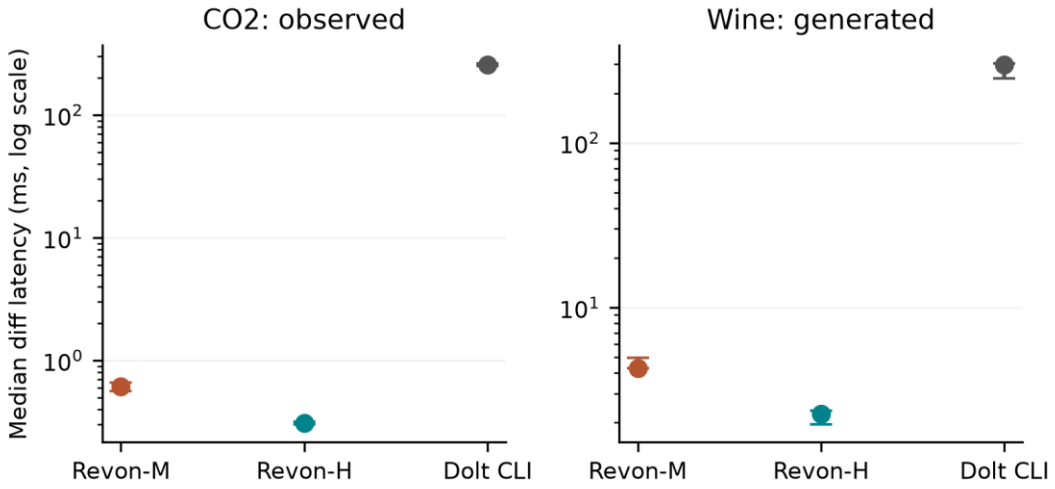


Fig 9. Workflows on public sources. Points show the median of three repository medians and bars show their full range. CO2 repositories repeat one observed publication history; wine repositories use three generated correction histories. Each repository contributes seven cached queries. Different API/CLI boundaries mean these are workflow latencies, not measurements of isolated storage engines.

In co2-observed, median Dolt command time is 256.063 ms, with 0.501 ms for JSON interpretation and 0.503 ms for common output construction. The command takes 99.6% of median complete diff latency. It includes startup, native work and native JSON emission, so the breakdown cannot assign that entire interval to the storage engine or justify subtracting a separately measured startup constant.

In wine-generated, median times are 288.464 ms for the Dolt command, 4.445 ms for JSON interpretation and 3.258 ms for common output construction. The command represents 97.2% of median complete diff latency and includes startup, native work and native JSON emission. As for CO2, these measurements neither isolate the engine's cost nor justify subtracting an independent startup estimate.

6.4. Repeated Windows scale observations.

Table 15
Windows scale repetitions: medians of three seed/repository summaries; full ranges are recorded in the evidence

Rows / system	Import s	Commit ms	Diff ms	Checkout s	Worker RSS GiB
100,000/revon-m	1.116	54.309	5.649	0.209	0.248
100,000/revon-h	1.109	53.044	1.004	0.173	0.242
100,000/dolt	1.662	759.99	283.65	0.662	0.223
1,000,000/revon-m	10.614	88.233	33.694	1.862	1.939
1,000,000/revon-h	10.738	85.362	1.013	1.575	1.925
1,000,000/dolt	11.281	890.97	320.30	3.706	0.976

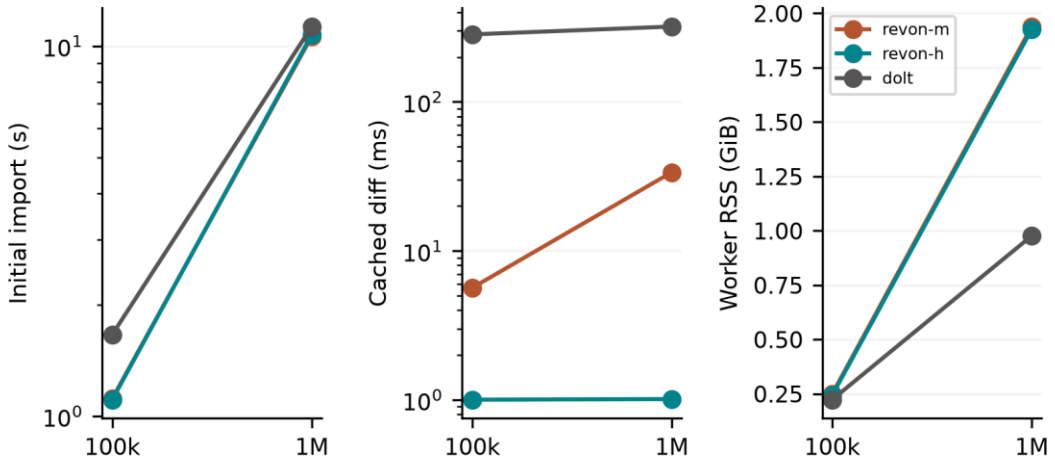


Fig 10. Repeated Windows scale observations with three generated histories and fresh repositories per size/system. Connecting lines show descriptive medians at two sizes, not fitted asymptotic laws. Diff queries use cached state; import and checkout are timed independently. Peak RSS covers workload/oracle generation, storage work, verification and sampled descendants. Linux million-row n=1 diagnostics are from a separate campaign.

We keep the repeated Windows scale analysis separate from the Linux diagnostics. Its three histories use different updates on the same initial synthetic distribution, rather than independent public datasets. The manifest records available memory and background processes, but those conditions are uncontrolled. The observations describe the workflows tested under those conditions; they do not establish a ranking across a population or a general scaling law.

6.5. Answers to the research questions. RQ1 is studied through the tradeoffs in latency, persistent storage, and workers involved in complete workflows. The boundary conditions of API and CLI make the influence of such tradeoffs impossible to interpret as an engine ranking only. As for RQ2, logging brings better results to a number of fixed history timings, but Windows calibration implements the strategy that prefers logging for all intervals being tested, so the benefit of switching remains undetermined. As for RQ3, it shows the influence of payload, routing, and geometry in staged design using controlled histories, but there is no common layout to be found. Transitions CO2 observed differ from wine corrections and synthetic updates.

7. Discussion. The workflow measurements answer Research Question 1 (RQ1) at the application interface. Measurements for import, commit, diff, checkout, and persistent footprint have been gathered in both the Linux and Windows experiments. Rows per campaign are considered individually. Both the snapshot and Log-only settings demonstrate representation costs; while the forced M, H, and log comparisons fix Revon's representation. Dolt provides a richer SQL/versioning interface. The separation into discrete command phases enables us to investigate the workflow without mixing it with tree operations and a standalone Python call.

Research Question 2 (RQ2) explores if the parameter selected for one set of histories can be applied to other sets of histories. In the Windows split, the primary parameters (N, H, payload, and geometry) will remain constant and the selection will be recorded prior to the validation phase. The largest candidate in this case is the always-log setting within the measured parameter space. Parity in dense cases and the unfavorable Linux threshold selection show why taking the fastest route is not possible. More sophisticated selection would require independent calibration followed by untainted measurement. No grounds for changing the default threshold have been found here.

Research Question 3 (RQ3) demonstrates that equal capacity does not necessarily result in identical traversal depths and costs of ancestor reconstruction even if the expected occupancy is identical with uniform routing. Payload encoding, leaf inspection, and output construction add time to the latency. Seeds are independently generated in the Windows cases with the controls, while the Linux geometry uses a single history. Neither configuration models all interactions nor finds the optimum for a population.

Consistent value semantics are necessary for diff comparison. Detached input and output of Revon prevent mutable aliases from affecting history retention. The v1 encoded identity value true is distinct from 1, 1 is distinct from 1.0, and object encoding and the on-disk schema is unchanged. The Linux test exercises these semantics against provided histories. Unit tests check for null vs. absence, nested ownership, unsupported values, and rollback.

8. Limitations And Future Work. External Validity: The Linux campaign was run on a single cloud virtual machine, whereas the Windows manifest was done on a single local machine, both controlled by the authors. Such setups cannot be considered independent party replications or operating system comparison. There are a number of co-variables associated with hardware, runtime, virtualization, and storage properties. Caches and background services were enabled and the Windows manifest reports limited amount of RAM available and process status. This study does not capture idle host or cold caches.

Workload Validity: TLC histories were generated with updates to one monthly release. Linux scale diagnostic runs are n=1 per system. The Windows validation uses three generated seed per primary profile, generated wine corrections and one observed CO2 publication sequence. Three CO2 repositories use the same sequence of events instead of generating independent histories. Header-only revisions are trimmed before timing, whereas unnamed columns are kept but are not scientifically analyzed. The synthetic scale histories are sampled from a common distribution. Broader generalization would require other domains, observations and hardware.

Implementation Scope: Revon trie uses a fixed geometry, eagerly loading the retained nodes, changesets and commits, without eviction. The fixed-geometry trie implementation supports single writer and linear history. The evaluation does not consider named branches, merge commits, conflict resolution, schema evolution, remote connections, ordered scans, crash faults injection or concurrent execution. The atomic SQLite transactions alone are not sufficient proof of all durability guarantees required for production.

Measurement and Inference: Dolt command time measures the whole process execution time, including native computations and JSON emission. Worker RSS measure includes oracle and history state. Sampling interval of ten milliseconds may miss the process peak or even process. Windows I/O counters measure neither physical disk usage nor database logical reads and the comparison between sampler on/off conditions confuse order and caching. Linux bootstrap time captures repeated timings of fixed histories. Windows three-history timings represent descriptive statistics and not adjusted for multiple comparisons. Checks of sources and semantic hashes are used for internal self-consistency, but not timing repeatability checks.

Selection and Design: the first Windows calibration examines four threshold settings, two starting seeds, and the selected profiles. Its highest test candidate, T=65,536, never switches in the selected test range, leaving the potential benefits of broader selection-based adaptation unclear. The sensitivity design is partial rather than full-factorial: for constant operations H is compared to density, and in hash route tests the appropriate low density control must be used. Another selection mechanism or crossover would require a separate constant experiment.

Future storage enhancements may involve binary nodes, object values, packed objects, data compression, and reachability-based garbage collection. An ordered Merkle index or content-driven chunking can facilitate a range operations alternative approach. Each change requires semantic and performance testing; the results for fixed tries cannot predict their influence.

9. Conclusion. Revon offers two semantically coherent diff paths on a content-addressed, fixed-trie history stored atomically in SQLite. The contribution represents an inspectable prototype supported by empirical evaluations of workflow and selection costs. Linux-based results include 516 executions supplemented with 72 executions and cover the dense case result in question, along with the negative threshold selection choice. Windows-based results consist of 93 worker tests and 3,402 query tests performed on held-out histories, Wine Quality and CO2 data sets, interface phases, and scaled runs. The tuned policy with T = 65,536 applies log all through the observed primary matrix, leaving the best crossover point still unknown. The documented workloads demonstrate conditional trade-offs, limited by the restrictions of API/CLI usage, sampling, data set, and hardware. These do not confirm a universal speed benefit and do not allow for independent reproduction of the results.

Funding: The authors received no funding for this study.

Competing interests: There are no competing interests to declare.

Ethics and consent to participate: This computational study recruited no participants and involved no human or animal intervention. It uses synthetic histories and public de-identified TLC records. The authors claim no institutional ethics approval or exemption.

Consent for publication: The paper contains no identifiable participant material. Consent for publication is therefore not applicable.

Data and code availability: Public inputs comprise NYC TLC taxi records [24], UCI Wine Quality [11] and the published CO2 package [12]. Code, executed source snapshots and experimental evidence are provided at https://github.com/atharvasheersh/Revon. The paper/ directory holds the Linux reproducibility archive and the Windows reproducibility archive. We do not redistribute TLC input records or generated databases. Source metadata records the licences and transformations; Appendix A explains how to access the materials.

Acknowledgment. We thank the open-source Dolt project and the researchers in dataset versioning whose work informed our experimental design.

REFERENCES

[1] M. Akhlaghi et al. (2021). Toward long-term and archivable reproducibility, Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, https://doi.org/10.1109/MCSE.2021.3072860. [Online]. Available: https://arxiv.org/abs/2006.03018.

[2] M. Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, Proc. VLDB Endowment, vol. 13, no. 12, pp. 3411-3424, https://doi.org/10.14778/3415478.3415560.

[3] Attic Labs. Noms Technical Overview. GitHub. [Online]. Available: https://github.com/attic-labs/noms/blob/master/doc/intro.md.

[4] A. Auvolat and F. Taïani (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS), pp. 221-230, https://doi.org/10.1109/SRDS47363.2019.00032.

[5] J. Benet (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: https://arxiv.org/abs/1407.3561.

[6] A. Bhardwaj et al. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR), Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.

[7] S. Bhattacherjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, Proc. VLDB Endowment, vol. 8, no. 12, pp. 1346-1357, https://doi.org/10.14778/2824032.2824035.

[8] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas (2020). Fully persistent B-trees, Theoretical Computer Science, vol. 841, pp. 10-26, https://doi.org/10.1016/j.tcs.2020.06.027. [Online]. Available: https://cs.au.dk/~gerth/papers/tcs20.pdf.

[9] P. Buneman, S. Khanna, and W.-C. Tan (2001). Why and where: A characterization of data provenance, in Database Theory: ICDT Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.

[10] S. Chacon and B. Straub (2014). Git Internals: Git Objects, in Pro Git, 2nd ed., [Online]. Available: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects.

[11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis (2009). Wine Quality [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C56S3T. CC BY 4.0.

[12] DataHub Contributors (2026). CO2 PPM - Trends in Atmospheric Carbon Dioxide, NOAA-derived published data package, accessed October https://github.com/datasets/co2-ppm. ODC-PDDL-1.0; acquired revision identifiers and file hashes are retained in the experiment manifest.

[13] DoltHub (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: https://www.dolthub.com/docs/architecture/storage-engine/block-store/.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan (1989). Making data structures persistent, J. Comput. Syst. Sci., vol. 38, no. 1, pp. 86-124, https://doi.org/10.1016/0022-0000(89)90034-2.

[15] J. Edwards and T. Petricek (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: https://arxiv.org/abs/2512.09762.

[16] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez (2024). Standardised versioning of datasets: A FAIR-compliant proposal, Scientific Data, vol. 11, art. 358, https://doi.org/10.1038/s41597-024-03153-y. [Online]. Available: https://www.nature.com/articles/s41597-024-03153-y.

[17] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: https://arxiv.org/abs/2609.02106.

[18] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, https://doi.org/10.1109/IPDPS57955.2024.00049. [Online]. Available: https://arxiv.org/abs/2402.11741.

[19] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, https://doi.org/10.14778/3115404.3115417.

[20] M. Maddox et al. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, https://doi.org/10.14778/2947618.2947619.

[21] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.

[22] R. C. Merkle (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.

[23] A. Muthitacharoen, B. Chen, and D. Mazières (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, https://doi.org/10.1145/502034.502052.

[24] New York City Taxi and Limousine Commission (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[25] N. Saquib, C. Krintz, and R. Wolski (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, https://doi.org/10.1109/TPDS.2022.3217969. [Online]. Available: https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf.

[26] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, https://doi.org/10.1145/3335783.3335792. [Online]. Available: https://www-db.cs.tum.edu/~schuele/data/tardis.pdf.

[27] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, https://doi.org/10.1109/ICDE.2012.102. [Online]. Available: https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f.

[28] T. Sehn (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: https://www.dolthub.com/blog/2024-02-29-storage-engine/.

[29] R. Shraga and R. J. Miller (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, https://doi.org/10.14778/3583140.3583169.

[30] E. Soroush and M. Balazinska (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, https://doi.org/10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.

[31] A. Twigg, A. Byde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: https://arxiv.org/abs/1103.4282.

[32] S. Wang et al. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, https://doi.org/10.14778/3231751.3231762.

[33] J. Zhao and S. Krishnan (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: https://arxiv.org/abs/2405.17701.

[34] J. Klump, H. Pampel, L. Rothfritz, and D. Strecker (2024). Recommendations on Data Versioning. Zenodo. https://doi.org/10.5281/zenodo.13743875

[35] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, et al. (2022). Design and evaluation of IPFS. Proceedings of the ACM SIGCOMM 2022 Conference, pp. 739-752. https://doi.org/10.1145/3544216.3544232

[36] A. Arasu, B. Chandramouli, J. Gehrke, E. Ghosh, D. Kossmann, J. Protzenko, et al. (2021). FastVer: Making Data Integrity a Commodity. Proceedings of the 2021 International Conference on Management of Data, pp. 89-101. https://doi.org/10.1145/3448016.3457312

[37] E.D. Demaine, S. Langerman, E. Price (2010). Confluently Persistent Tries for Efficient Version Control. Algorithmica, vol. 57, no. 3, pp. 462-483. https://doi.org/10.1007/S00453-008-9274-Z

[38] S. Lanka, E. Mays (1991). Fully persistent B+-trees. ACM SIGMOD Record, vol. 20, no. 2, pp. 426-435. https://doi.org/10.1145/119995.115861

[39] G.S. Brodal, C.M. Rysgaard, R. Svenning (2023). External Memory Fully Persistent Search Trees. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, pp. 1410-1423. https://doi.org/10.1145/3564246.3585140

[40] C. Allain, B. Clément, A. Moine, G. Scherer (2024). Snapshottable Stores. Proceedings of the ACM on Programming Languages, vol. 8, no. ICFP, pp. 338-369. https://doi.org/10.1145/3674637

[41] A. Miller, M. Hicks, J. Katz, E. Shi (2014). Authenticated data structures, generically. Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 411-423. https://doi.org/10.1145/2535838.2535851

[42] V. Salis, D. Spinellis (2019). RepoFS: File system view of Git repositories. SoftwareX, vol. 9, pp. 288-292. https://doi.org/10.1016/j.softx.2019.03.007

[43] A. Bhardwaj, A. Deshpande, A.J. Elmore, D. Karger, S. Madden, A. Parameswaran, et al. (2015). Collaborative data analytics with DataHub. Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1916-1919. https://doi.org/10.14778/2824032.2824100

[44] A. Chavan, A. Deshpande (2017). DEX. Proceedings of the 2017 ACM International Conference on Management of Data, pp. 171-186. https://doi.org/10.1145/3035918.3064056

[45] J.F. Roddick (1995). A survey of schema versioning issues for database systems. Information and Software Technology, vol. 37, no. 7, pp. 383-393. https://doi.org/10.1016/0950-5849(95)91494-K

[46] G. Ozsoyoglu, R. Snodgrass (1995). Temporal and real-time databases: a survey. IEEE Transactions on Knowledge and Data Engineering, vol. 7, no. 4, pp. 513-532. https://doi.org/10.1109/69.404027

[47] C. De Castro, F. Grandi, M.R. Scalas (1997). SCHEMA VERSIONING FOR MULTITEMPORAL RELATIONAL DATABASES. Information Systems, vol. 22, no. 5, pp. 249-290. https://doi.org/10.1016/S0306-4379(97)00017-3

[48] T.J. Green, G. Karvounarakis, V. Tannen (2007). Provenance semirings. Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 31-40. https://doi.org/10.1145/1265530.1265535

[49] Y.L. Simmhan, B. Plale, D. Gannon (2005). A survey of data provenance in e-science. ACM SIGMOD Record, vol. 34, no. 3, pp. 31-36. https://doi.org/10.1145/1084805.1084812

[50] L. Carata, S. Akoush, N. Balakrishnan, T. Bytheway, R. Sohan, M. Seltzer, et al. (2014). A primer on provenance. Communications of the ACM, vol. 57, no. 5, pp. 52-60. https://doi.org/10.1145/2596628

[51] B. Glavic (2021). Data Provenance. Foundations and Trends in Databases, vol. 9, no. 3-4, pp. 209-441. https://doi.org/10.1561/1900000068

[52] Y. Amsterdamer, D. Deutch, V. Tannen (2011). Provenance for aggregate queries. Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 153-164. https://doi.org/10.1145/1989284.1989302

[53] F. Jamil, A. Khan, A. Anjum, M. Ahmed, F. Jabeen, N. Javaid (2018). Secure provenance using an authenticated data structure approach. Computers & Security, vol. 73, pp. 34-56. https://doi.org/10.1016/j.cose.2017.10.005

[54] M.D. Wilkinson, M. Dumontier, I.J. Aalbersberg, G. Appleton, M. Axton, A. Baak, et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, vol. 3, no. 1, article 160018. https://doi.org/10.1038/sdata.2016.18

[55] B. Pan, N. Stakhanova, S. Ray (2023). Data Provenance in Security and Privacy. ACM Computing Surveys, vol. 55, no. 14s, pp. 1-35. https://doi.org/10.1145/3593294

[56] L. Moreau, P. Groth, J. Cheney, T. Lebo, S. Miles (2015). The rationale of PROV. Journal of Web Semantics, vol. 35, pp. 235-257. https://doi.org/10.1016/j.websem.2015.04.001

[57] K. Wolstencroft, S. Owen, O. Krebs, Q. Nguyen, N.J. Stanford, M. Golebiewski, et al. (2015). SEEK: a systems biology data and model management platform. BMC Systems Biology, vol. 9, no. 1, article 33. https://doi.org/10.1186/s12918-015-0174-y

[58] D. Comer (1979). Ubiquitous B-Tree. ACM Computing Surveys, vol. 11, no. 2, pp. 121-137. https://doi.org/10.1145/356770.356776

[59] Y.S. Kwong, D. Wood (1980). On B-trees. Proceedings of the 1980 ACM SIGMOD international conference on Management of data - SIGMOD '80, pp. 207. https://doi.org/10.1145/582250.582282

[60] W.J. Hansen (1981). A Cost Model for the Internal Organization of B+-Tree Nodes. ACM Transactions on Programming Languages and Systems, vol. 3, no. 4, pp. 508-532. https://doi.org/10.1145/357146.357152

[61] M.A. Bender, E.D. Demaine, M. Farach-Colton (2005). Cache-Oblivious B-Trees. SIAM Journal on Computing, vol. 35, no. 2, pp. 341-358. https://doi.org/10.1137/S0097539701389956

[62] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil (1996). The log-structured merge-tree (LSM-tree). Acta Informatica, vol. 33, no. 4, pp. 351-385. https://doi.org/10.1007/s002360050048

[63] F. Chang, J. Dean, S. Ghemawat, W.C. Hsieh, D.A. Wallach, M. Burrows, et al. (2008). Bigtable. ACM Transactions on Computer Systems, vol. 26, no. 2, pp. 1-26. https://doi.org/10.1145/1365815.1365816

[64] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, et al. (2007). Dynamo. Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, pp. 205-220. https://doi.org/10.1145/1294261.1294281

[65] M. Rosenblum, J.K. Ousterhout (1992). The design and implementation of a log-structured file system. ACM Transactions on Computer Systems, vol. 10, no. 1, pp. 26-52. https://doi.org/10.1145/146941.146943

[66] L. Lu, T.S. Pillai, H. Gopalakrishnan, A.C. Arpaci-Dusseau, R.H. Arpaci-Dusseau (2017). WiscKey. ACM Transactions on Storage, vol. 13, no. 1, pp. 1-28. https://doi.org/10.1145/3033273

[67] H.T. Vo, S. Wang, D. Agrawal, G. Chen, B.C. Ooi (2012). LogBase. Proceedings of the VLDB Endowment, vol. 5, no. 10, pp. 1004-1015. https://doi.org/10.14778/2336664.2336673

[68] C. Riegger, I. Petrov (2024). Storage Management with Multi-Version Partitioned BTrees. Information Systems, vol. 125, pp. 102403. https://doi.org/10.1016/j.is.2024.102403

APPENDIX A: REPRODUCIBILITY

The repository for the Revon project provides the source code and experiment logs related to this research. The Linux reproducibility archives and Windows reproducibility archives are both found in the paper/ folder. READMEs provide instructions on dependency setup, public data collection, conducting experiments, and verifying results [1]. Each pack includes the source code used, protocol followed, data gathered, and internal audit.

The main Linux campaign, the supplementary Linux campaign, and the Windows campaign utilize different protocols and must be analyzed separately. Instructions for preparation are provided in order to recreate the inputs for the TLC as well as the created repositories. Metadata about the public source licensing, data collection version, checksums, and CO2 removal before the timing is provided. In order to keep the collected logs intact, the auditors should work on the disposable copy with the local data paths. Semantic hash comparison is used to ensure consistency, however, this does not result in the same elapsing time and identical timings on a different machine.

APPENDIX B: METHODS AND EVIDENCE DETAILS

Linux tarballs keep the sources that have been executed, along with their hashes, environments, fixed workloads, order of execution, observations, and audit trail. The workers start with PYTHONHASHSEED=20261008. Preflight runs 67 unit tests for the main and 69 for the supplemental campaign before measurement. The preflight of the supplemental campaign reports an incomplete data transfer due to a missing test_data directory. After the fixture directory is restored, the entire test suite runs successfully before the time measurements start. In case of setup failure, the execution logs keep the record.

Each run of the main campaign creates a repository in the new worker. Deterministic shuffling and rotation decide on the order of models within each repetition block. Import timings stop on the first durable version. The median of individual commit timers gives the incremental commit latency of a particular history. Diff produces the comparison of the initial and final versions in the form of sorted changed keys with their old/new values' hashes in canonical JSON format. The checkout writes the entire sorted key-value state in canonical UTF-8 JSON. The Revon engine is run using in-process API; Dolt timings include the invocation of the command line interface, JSON parsing, and output

normalization. Therefore, workflow ratios cannot be compared in terms of the speed of engines.

Correctness verification is excluded from the operation timings. For each run of the main campaign, we verify the initial and final state hashes, all H+1 historical states, and the observed canonical diff against the workload oracle built separately. Verification logs include the expected and observed hashes. Audit checks regenerate workloads, check matrix membership, selected paths and source/data/protocol identities, and compute summaries. Downloaded logs are audited too. These checks guarantee internal integrity, not the independent replication of the timing experiment.

The medians and IQRs of latency for the seven examined repository trials after excluding the warm-up period are presented below. The latency ratios are calculated by comparing the matched pairs of the trials. For each pair of trials, the ratio's median value and paired percentile-bootstrap confidence interval with 20,000 bootstrap resamples are reported. A ratio higher than one implies preference for the denominator. The exploratory 95% confidence intervals refer to repeated timing of the same workload on one virtual machine and are neither representative of population effect nor adjusted for multiple comparisons. The median of paired ratios does not coincide with the ratio of latency medians. All finished trials are kept; timing outliers are included too.

In order to sample the resident set size (RSS), CPU utilization, and I/O usage of the worker process and its descendents, the parent process collects information every 10 milliseconds. In addition to storage operations, peaks contain information about workload creation, oracle state, and verification. Therefore, it is impossible to attribute the whole peak to the storage engine. Short-lived subprocesses or temporary peaks can be observed between samples. The main Linux sampler considers a fully zero read counter to be unavailable data and cannot provide any information about read I/O. The second campaign distinguishes measured zeros and unavailable counters, so it is analyzed separately. Cached bytes can be logically analyzed while physical read bytes are zero.

The Linux footprint measure represents the directory size before the closure and compaction. It also includes indexes, metadata, and representation cost. Logical-payload normalization and compaction probes of the first trial are included in the measurements. The measurements relate to the complete workflow and not just tree overhead. CPU, RSS, and I/O measurements cover the whole life cycle of the worker process and last longer than operation timers.

In the auxiliary Windows threshold experiment, the calibration and validation profiles are pre-defined and non-overlapping. Possible candidate values of T are 1,024, 2,048, 4,096, 8,192, and 16,384 operations. The minimization of normalized calibration latency leads to the selection of T = 16,384 prior to validation, resulting in a held-out normalized score of 1.032 compared to 1.297 for T = 4,096. The always-log scores are 1.000 in both splits. The cached-query data correspond to the recorded source and environment of the experiment and are distinct from Linux measurements. One largest-candidate winner does not form an optimum. Long-horizon history and three seeds pilot create the exploratory setting.

Linux threshold control uses a fixed value of T = 16,384 with 10,000 rows and 512 changes per commit. Values H = 16 and H = 64 lead to 8,192 and 32,768 operations, respectively. Forced log, forced Merkle, and hybrid configurations run separately, each using two warm-up runs and seven measured trials. The threshold is not based on Linux results. These cases assess the transfer of the auxiliary calibration choice without going through another calibration/validation cycle.

In Linux geometry, the fixed setup is 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values, and seed 20260824. Forced Merkle diffs are applied for b4-d6, b8-d3, b8-d4, b8-d5, and b16-d3. Every configuration uses separate workers for two warm-up runs and seven measured trials, rotating the execution order to prevent time/configuration confounding. Every trial checks eleven historical states and the endpoint diff outside of timing measurement. The output contract corresponds to the workflow experiment. Peaks of geometry adapter observed by the in-process monitor are separated from external RSS measurements.

The Windows protocol and case manifest specify calibration seeds 202610161/202610162 and disjoint validation/sensitivity seeds 202610171-202610173. Workers use PYTHONHASHSEED=20261016. Monitoring uses seed 202610190, generated public corrections use 202610201-202610203, and scale uses 202610221-202610223. Each worker builds fresh persistent state, but its queries share one repository. We timestamp the threshold before validation and do not repeat selection afterwards. Linux/Windows rows are analysed separately.

Windows instrumentation runs the native Dolt JSON-diff command under the adapter's parser contract and logs explicit phase timers. Preflight comparison requires the evaluation of the observed output against the native adapter as well as the oracle. Revon uses the hash comparisons of historical states, hashes of all historical states, reverse/identity requests, object integrity, and re-opening outside the established timing to validate the observed output. Setup validation and data-quality exceptions are performed before measurement and logged. The captured query matrix and timing source hashes establish the run-time environment.

Portable audit relies on the disposable copy of the evidence. In a new interpreter, it loads the executed source code and determines the public-data path from the logs. It regenerates the workloads and state/diff oracles, validates each configuration, trial, and order identity, and computes the numerical summary. Hashes of collected files are verified next. These validations and data acquisition routines are performed in the archives for consistency validation; replication of timing requires the real experiment execution.

Table 16
Checksums of public data and the Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

Table 17
Windows execution matrix; cached queries are counted separately from independent history or repository repetitions

Component	Fresh workers	Histories / control	Warm-up queries	Measured queries
Calibration	12	2 seeds × 2 localities × 3 densities	144	504
Held-out	18	3 different seeds × same profiles	216	756
Sensitivity	21	3 seeds × 7 matched variations	252	882
Monitor diagnostic	6	3 off/on pairs, one fixed history	72	252
Public sources	18	2 sources × 3 runs × M/H/Dolt	36	126
Scale	18	2 sizes × 3 seeds × M/H/Dolt	36	126
Total	93	Different campaigns are never pooled	756	2646